TRIBHUVAN UNIVERSITY

INSTITUTE OF SCIENCE AND TECHNOLOGY

AMRIT SCIENCE CAMPUS

Advanced Java Programming (CSC 419)

Seventh Semester Lab Report

SUBMITTED BY:

Name: Samchita Pandey
Roll: 79010245
Date: 2083/01/28

SUBMITTED TO:

Chakra Narayan Rawal
Department of CSIT

External Signature

Internal Signature

CORE JAVA

Table of Contents

S.N	Title	Date	Signature
1	Write a Java program that takes two numbers as input and display the sum, product, quotient, remainder and average of two numbers.		
2	Write a program to check whether the input number is odd or even.		
3	Write a program that takes length and breadth as input and calculate and display area of a rectangle.		
4	Write a program to find sum of digits of a number (e.g. 123 = 1+2+3 = 6).		
5	Write a program to find the sum of even and odd numbers from 1 to 100.		
6	Write a program to check the given number is palindrome or not.		
7	Write a program to find sum of elements of an array.		
8	WAP to input n elements of an array and sort them in ascending/descending order.		
9	Write a program to input and calculate sum of two matrices.		
10	Write a program to input and calculate product of m×n matrices.		
11	Create a class Box with instance variables length, breadth and height. Add getVolume() method. Create subclass BoxWeight using super keyword. Create BoxDemo with main() to create two BoxWeight objects and display volumes and weights.		
12	Create two classes Thread A and Thread B which implement Runnable interface. Thread A displays all even numbers from 50 to 100 and Thread B displays all odd numbers from 100 to 200.		

S.N	Title	Date	Signature
13	Write a program to show handling multiple exceptions using multi-catch and finally.		
14	Create a package "Area" to calculate and display area of rectangle, circle and square. Write a program to display area using package "Area".		
15	Write a program to take employee id, name, address, DOB and phone number from user and store it in a file called "Emp.txt". Also display the contents of "Emp.txt".		
16	Write a program to read name, age, grade, contact number of student and write the record in file std.txt and copy the content to std1.txt then display the content of std1.txt.		

ADVANCED JAVA

Table of Contents

S.N	Title	Page No	Date	Signature
1	Write a program using AWT components to add two numbers. Use text fields for inputs and output. Display result when user presses a button.	1-2		
2	Write a program using swing components to multiply two numbers. Use text fields for inputs and output. Display result when user presses a button.	2-4		
3	Write a program to illustrate the use of BorderLayout.	5-6		
4	Create a login form with username and password fields. Print "access granted" if both are "admin", else print "access denied".	7-8		
5	Create a basic notepad app with features: New, Open, Save, Exit using JButton components.	9-11		
6	Create a form using JFrame to collect student records: First Name (JTextField), Last Name (JTextField), Age (JTextField), Gender (JRadioButton), Faculty (JComboBox), Semester (JComboBox), Remarks (JTextArea).	12-15		
7	Write a Java code to connect database and perform insert, select, delete and update (CRUD) operations in database table.	16-17		
8	Write a program using Connection oriented Socket Programming that can read and write on both sides.	18-21		
9	Write client and server programs where server accepts radius from client, computes area, sends computed area to client, and client displays it.	22-24		

S.N	Title	Page No	Date	Signature
10	Write program for client server application using TCP and UDP.	25-29		
11	Write a program that illustrates Handling HTTP Request and Response using servlets.	30-31		
12	Write a simple JSP program to display "Tribhuvan University" 10 times.	32		
13	Write a program to create a JSP web form to take input of a student and submit it to a second JSP file which prints the values of form submission.	33-34		
14	Write a program using RMI to find sum and difference of two numbers. Methods sum and difference should be invoked from a remote machine.	35-37		
15	Program that illustrates the example of servlet and database.	38-41		
16	Program illustrating example of scriptlet tag, declaration tag, and expression tag.	42-43		

LAB 1

OBJECTIVE: To write a program using components to add two number and use text fields for inputs and output that displays result when user press a button.

THEORY

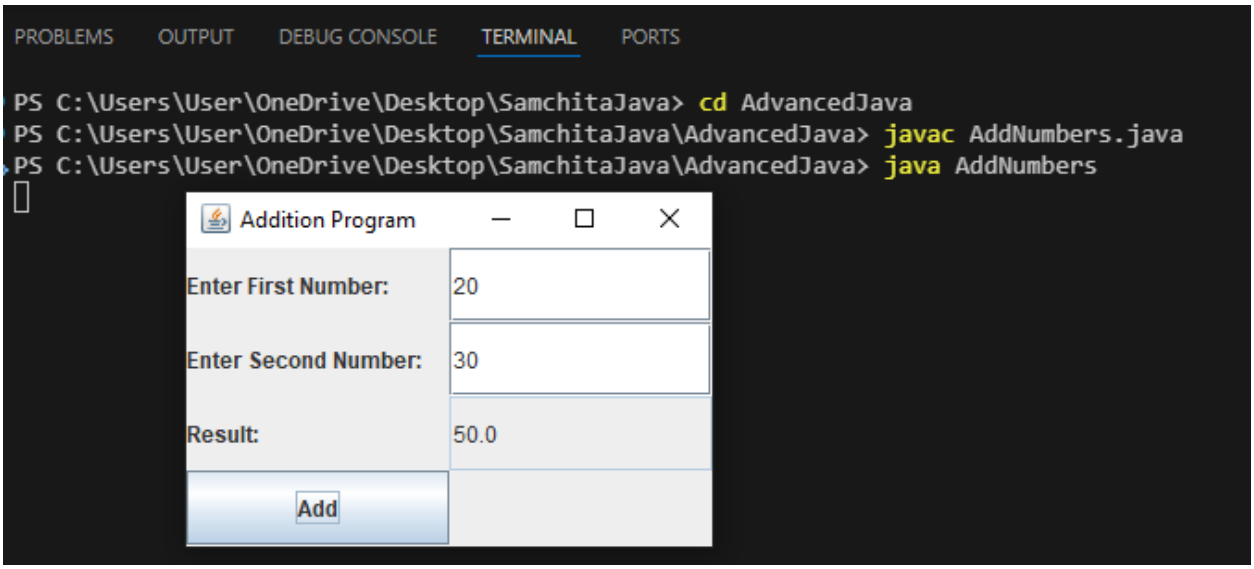
Java Swing is a GUI (Graphical User Interface) toolkit used to create desktop applications. In this program, JFrame is used as the main window, JLabel is used for displaying text, JTextField is used for user input and output, and JButton is used to trigger the addition operation. The program implements ActionListener so that when the button is clicked, the actionPerformed() method executes and calculates the sum of two numbers entered by the user.

SOURCE CODE:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class AddNumbers extends JFrame implements ActionListener {
    JLabel l1, l2, l3;
    JTextField t1, t2, t3;
    JButton btn;

    AddNumbers() {
        setTitle("Addition Program");
        setSize(300,200);
        setLayout(new GridLayout(4,2));
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        l1 = new JLabel("Enter First Number:");
        t1 = new JTextField();

        l2 = new JLabel("Enter Second Number:");
        t2 = new JTextField();

        l3 = new JLabel("Result:");
        t3 = new JTextField();
|        t3.setEditable(false);

        btn = new JButton("Add");
        btn.addActionListener(this);

        add(l1); add(t1);
        add(l2); add(t2);
        add(l3); add(t3);
        add(btn);
```

```
        setVisible(true);
    }

    public void actionPerformed(ActionEvent e) {
        double num1 = Double.parseDouble(t1.getText());
        double num2 = Double.parseDouble(t2.getText());
        double result = num1 + num2;

        t3.setText(String.valueOf(result));
    }

    public static void main(String[] args) {
        new AddNumbers();
    }
}
```

OUTPUT



CONCLUSION

Thus, the program successfully demonstrated the use of Swing components, event handling, and text fields to perform addition of two numbers in Java GUI programming.

LAB 2

OBJECTIVE: To write a program using swing components to multiply two numbers. Use text fields for inputs and output that displays the result when the user presses a button.

THEORY

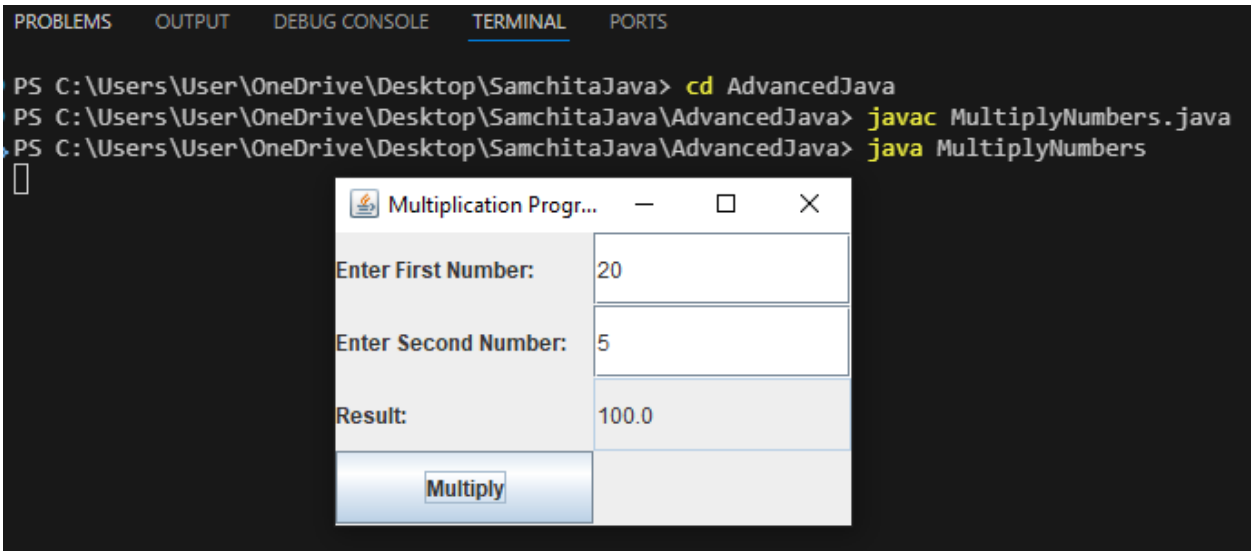
This program uses Java Swing components to create a simple multiplication calculator. The user enters two numbers in text fields and clicks the Multiply button. The ActionListener interface handles the button click event, retrieves input values, multiplies them, and displays the result in another text field.

SOURCE CODE

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class MultiplyNumbers extends JFrame implements ActionListener {
   JLabel l1, l2, l3;
   JTextField t1, t2, t3;
   JButton btn;

   MultiplyNumbers() {
      setTitle("Multiplication Program");
      setSize(300,200);
      setLayout(new GridLayout(4,2));
      setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

      l1 = new JLabel("Enter First Number:");”
      t1 = new JTextField();

     l2 = new JLabel("Enter Second Number:");
      t2 = new JTextField();

      l3 = new JLabel("Result:");
      t3 = new JTextField();

      t3.setEditable(false);

     btn = new JButton("Multiply");
      btn.addActionListener(this);

      add(l1); add(t1);
      add(l2); add(t2);
      add(l3); add(t3);
```

```
        add(btn);

        setVisible(true);
    }

    public void actionPerformed(ActionEvent e) {
        double num1 = Double.parseDouble(t1.getText());
        double num2 = Double.parseDouble(t2.getText());

        double result = num1 * num2;
        t3.setText(String.valueOf(result));
    }

    public static void main(String[] args) {
        new MultiplyNumbers();
    }
}
```

OUTPUT



CONCLUSION

Thus, the program successfully illustrated the use of Swing GUI components and event handling to perform multiplication of two numbers.

LAB 3

OBJECTIVE: To write a program to illustrate the use of BorderLayout.

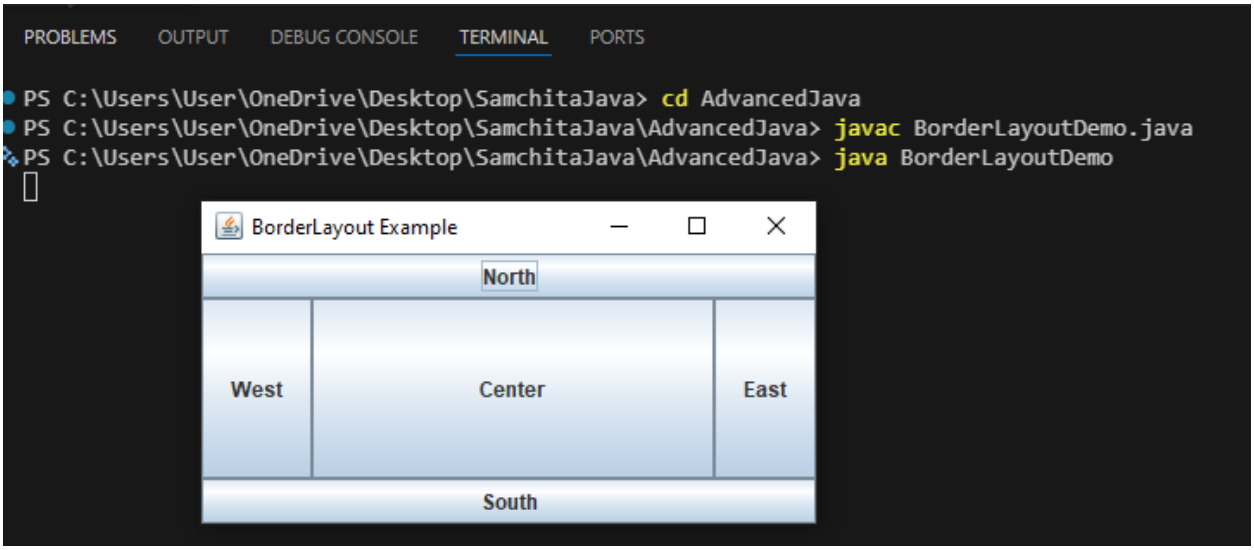
THEORY

BorderLayout is one of the layout managers in Java AWT/Swing. It divides the container into five regions: North, South, East, West, and Center. Components added to these regions automatically adjust according to the container size.

SOURCE CODE:

```
import javax.swing.*;
import java.awt.*;

public class BorderLayoutDemo extends JFrame {
   BorderLayoutDemo() {
      setTitle("BorderLayout Example");
      setSize(400,300);
      setLayout(new BorderLayout());
      setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
      add(new JButton("North"), BorderLayout.NORTH);
      add(new JButton("South"), BorderLayout.SOUTH);
     add(new JButton("East"), BorderLayout.EAST);
      add(new JButton("West"), BorderLayout.WEST);
      add(new JButton("Center"), BorderLayout.CENTER);
      setVisible(true);
   }
   public static void main(String[] args) {
      new BorderLayoutDemo();
   }
}
```

OUTPUT:



CONCLUSION

Thus, the program successfully demonstrated the use of BorderLayout manager for arranging components in different regions of a Java window.

LAB 5

OBJECTIVE: To create a login form with username and password fields that prints “access granted” if the username and password both are “admin”, when user clicks on Login button. If authentication fails, print “access denied”.

THEORY

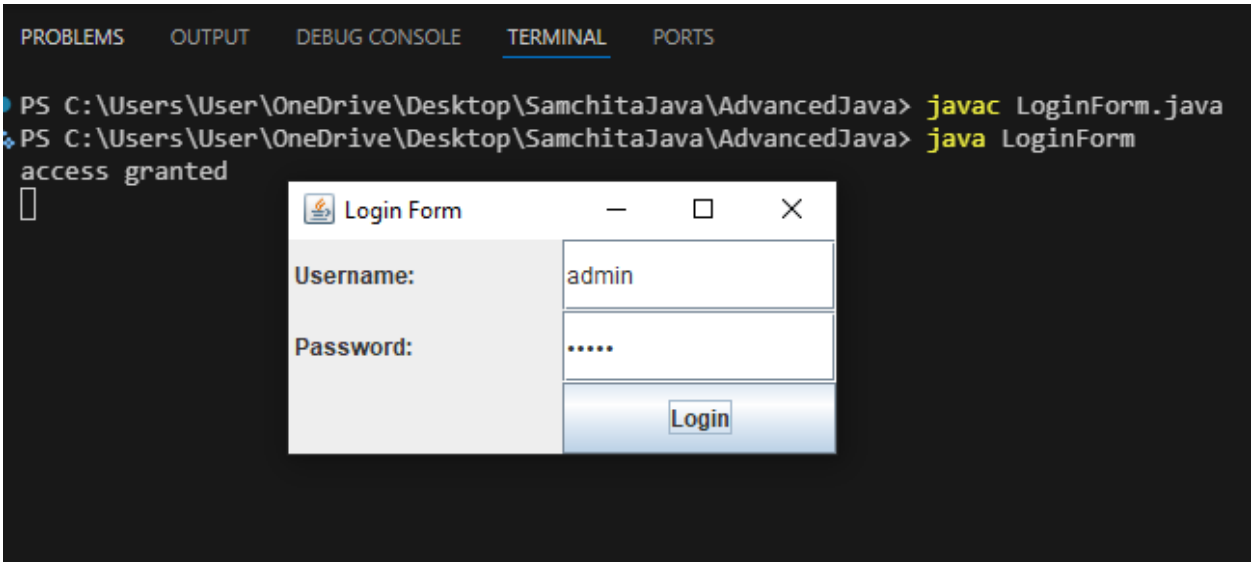
This program creates a simple login form using Swing components such as JTextField, JPasswordField, and JButton. The entered username and password are checked against predefined values (admin). Based on authentication, either “access granted” or “access denied” is displayed.

SOURCE CODE:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class LoginForm {
    public static void main(String[] args) {
        // Create the frame (window)
        JFrame frame = new JFrame("Login Form");
        frame.setSize(300, 150);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        frame.setLayout(new GridLayout(3, 2));

        // Create components
        JLabel userLabel = new JLabel(" Username:");
        JTextField userField = new JTextField();

         JLabel passLabel = new JLabel(" Password:");
        JPasswordField passField = new JPasswordField();
        JButton loginButton = new JButton("Login");

       // Add components to frame
        frame.add(userLabel);
        frame.add(userField);
        frame.add(passLabel);
        frame.add(passField);
        frame.add(new JLabel("")); // Empty space for layout
        frame.add(loginButton);

       // Add action listener to button
        loginButton.addActionListener(new ActionListener() {
            @Override
```

```
            public void actionPerformed(ActionEvent e) {
                String username = userField.getText();
                String password = new String(passField.getPassword());

                // Authentication logic
                if (username.equals("admin") && password.equals("admin")) {
                    System.out.println("access granted");
                } else {
                    System.out.println("access denied");
                }
            }
        });

        // Display the window
        frame.setVisible(true);
    }
}
```

OUTPUT



CONCLUSION

Thus, the program successfully demonstrated form creation, password handling, and simple authentication using Java Swing.

LAB 5

OBJECTIVE: To create a basic notepad app with the following features: a) New, b) Open, c) Save, d) Exit and use JButton components to implement these features.

THEORY

This program implements a basic text editor using Swing components. JTextArea is used for writing text, while buttons such as New, Open, Save, and Exit provide file handling operations. JFileChooser helps users select files for opening and saving.

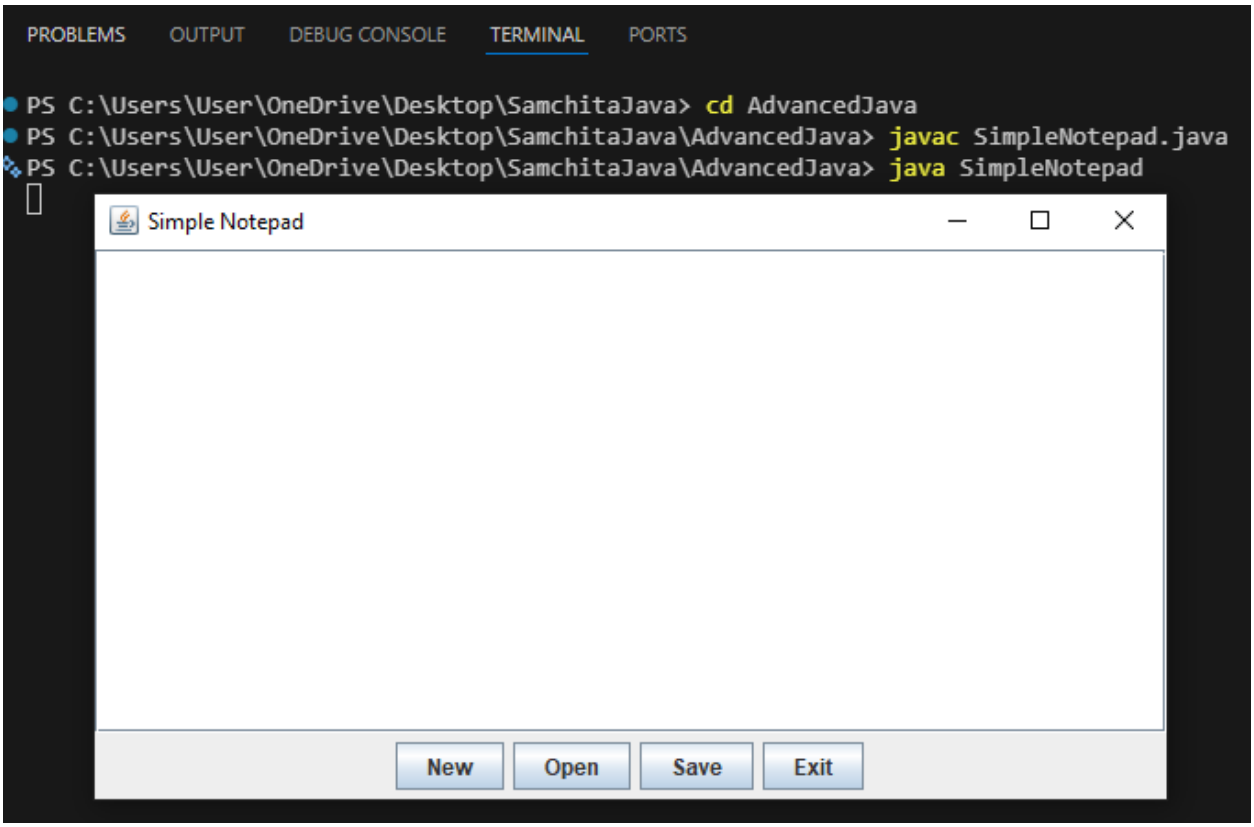
SOURCE CODE:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.io.*;

public class SimpleNotepad extends JFrame implements ActionListener {
    JTextArea textArea;
    JButton newButton, openButton, saveButton, exitButton;
    JFileChooser fileChooser;

    public SimpleNotepad() {
        // Frame setup
        setTitle("Simple Notepad");
        setSize(600, 500);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLayout(new BorderLayout());

        // Text Area
        textArea = new JTextArea();
        add(new JScrollPane(textArea), BorderLayout.CENTER);
        // Buttons Panel
        JPanel buttonPanel = new JPanel();
        newButton = new JButton("New");
        openButton = new JButton("Open");
        saveButton = new JButton("Save");
        exitButton = new JButton("Exit");

        // Add action listeners
        newButton.addActionListener(this);
        openButton.addActionListener(this);
        saveButton.addActionListener(this);
        exitButton.addActionListener(this);
```

```
    buttonPanel.add(newButton);
    buttonPanel.add(openButton);
    buttonPanel.add(saveButton);
    buttonPanel.add(exitButton);

    add(buttonPanel, BorderLayout.SOUTH);
    fileChooser = new JFileChooser();
 }

@Override
 public void actionPerformed(ActionEvent e) {
    if (e.getSource() == newButton) {
        textArea.setText("");
    } else if (e.getSource() == openButton) {
        int returnValue = fileChooser.showOpenDialog(null);
        if (returnValue == JFileChooser.APPROVE_OPTION) {
            File selectedFile = fileChooser.getSelectedFile();
            try (BufferedReader br = new BufferedReader(new FileReader(selectedFile))) {
                textArea.read(br, null);
            } catch (IOException ex) {
                ex.printStackTrace();
            }
        }
    } else if (e.getSource() == saveButton) {
        int returnValue = fileChooser.showSaveDialog(null);
        if (returnValue == JFileChooser.APPROVE_OPTION) {
            File fileToSave = fileChooser.getSelectedFile();
            try (BufferedWriter bw = new BufferedWriter(new FileWriter(fileToSave))) {
                textArea.write(bw);
            } catch (IOException ex) {
                ex.printStackTrace();
            }
        }
    } else if (e.getSource() == exitButton) {
        System.exit(0);
    }
 }

 public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        new SimpleNotepad().setVisible(true);|
    });
```

```
    }
}
```

OUTPUT


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\User\OneDrive\Desktop\SamchitaJava> cd AdvancedJava
PS C:\Users\User\OneDrive\Desktop\SamchitaJava\AdvancedJava> javac SimpleNotepad.java
PS C:\Users\User\OneDrive\Desktop\SamchitaJava\AdvancedJava> java SimpleNotepad
Simple Notepad
New
Open
Save
Exit

LAB 6

OBJECTIVE: To create a form using JFrame to collect the records of students in Asian College where each record should contain the following information:

a) First Name (JTextField)
b) Last Name (JTextField)
c) Age (JTextField)
d) Gender (JRadioButton)
e) Faculty (JComboBox/JList)
f) Semester (JComboBox/IList)
g) Remarks (JTextArea)

THEORY

This program creates a student registration form using different Swing components. JTextField is used for text input, JRadioButton for gender selection, JComboBox for faculty and semester selection, and JTextArea for remarks. GridBagLayout is used for arranging components systematically.

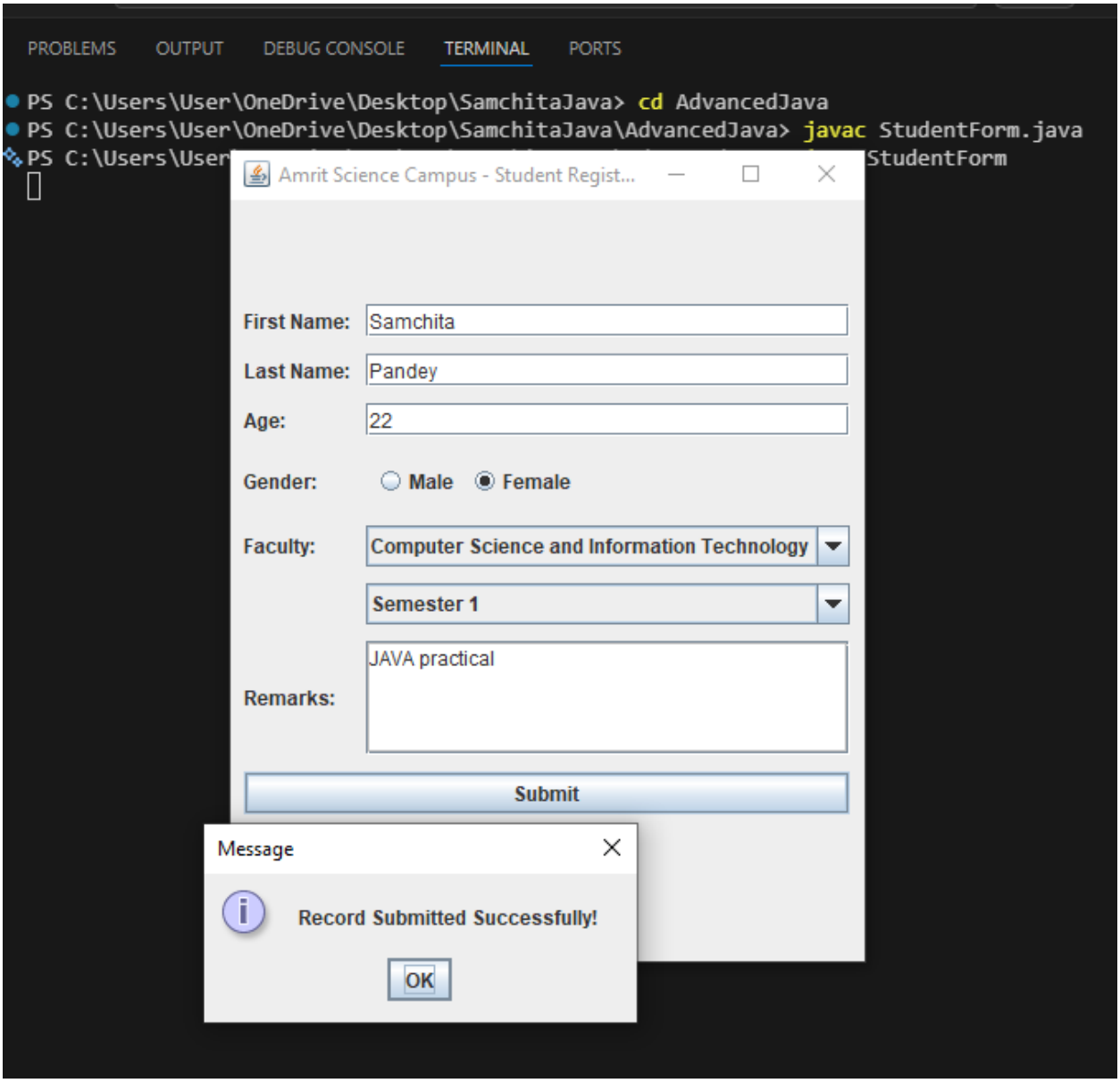
SOURCE CODE:

```
import javax.swing.*;
import java.awt.*;
public class StudentForm extends JFrame {

    private JTextField firstNameField, lastNameField, ageField;
    private JRadioButton maleRadio, femaleRadio;
    private JComboBox<String> facultyCombo, semesterCombo;
    private JTextArea remarksArea;
    private JButton submitButton;

    public StudentForm() {
        // Frame Setup
        setTitle("Amrit Science Campus - Student Registration");
        setSize(400, 500);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLocationRelativeTo(null); // Center on screen

        // Use GridBagLayout for a structured form
        setLayout(new GridBagLayout());
        GridBagConstraints gbc = new GridBagConstraints();
        gbc.insets = new Insets(5, 5, 5, 5); // Spacing
        gbc.fill = GridBagConstraints.HORIZONTAL;

        // --- Form Components ---
        // a) First Name
```

```
addComponent(new JLabel("First Name:"), 0, 0, gbc);
firstNameField = new JTextField(15);
addComponent(firstNameField, 0, 1, gbc);

// b) Last Name
addComponent(new JLabel("Last Name:"), 1, 0, gbc);
lastNameField = new JTextField(15);
addComponent(lastNameField, 1, 1, gbc);

// c) Age
addComponent(new JLabel("Age:"), 2, 0, gbc);
ageField = new JTextField(15);
addComponent(ageField, 2, 1, gbc);

// d) Gender (Radio Buttons)
addComponent(new JLabel("Gender:"), 3, 0, gbc);
JPanel genderPanel = new JPanel(new FlowLayout(FlowLayout.LEFT));
maleRadio = new JRadioButton("Male");
femaleRadio = new JRadioButton("Female");
ButtonGroup genderGroup = new ButtonGroup();
genderGroup.add(maleRadio);
genderGroup.add(femaleRadio);
genderPanel.add(maleRadio);
genderPanel.add(femaleRadio);
addComponent(genderPanel, 3, 1, gbc);

// e) Faculty
addComponent(new JLabel("Faculty:"), 4, 0, gbc);
String[] faculties = {"Computer Science and Information Technology", "B.Sc", "BIT"};
facultyCombo = new JComboBox<>(faculties);
addComponent(facultyCombo, 4, 1, gbc);

// f) Semester
addComponent(new JLabel("Semester:"), 8, 0, gbc);
String[] semesters = {"Semester 1", "Semester 2", "Semester 3", "Semester 4", "Semester
5", "Semester 6", "Semester 7", "Semester 8"};
semesterCombo = new JComboBox<>(semesters);
addComponent(semesterCombo, 5, 1, gbc);

// g) Remarks
addComponent(new JLabel("Remarks:"), 6, 0, gbc);
remarksArea = new JTextArea(4, 15);
remarksArea.setBorder(BorderFactory.createLineBorder(Color.LIGHT_GRAY));
```

```
        JScrollPane scrollPane = new JScrollPane(remarksArea);
        addComponent(scrollPane, 6, 1, gbc);

        // Submit Button
        submitButton = new JButton("Submit");
        gbc.gridx = 0;
        gbc.gridy = 7;
        gbc.gridwidth = 2; // Span across two columns
        add(submitButton, gbc);|

        // Basic Action Listener
        submitButton.addActionListener(e -> {
            JOptionPane.showMessageDialog(this, "Record Submitted Successfully!");
        });
    }

    // Helper method to add components to GridBagLayout
    private void addComponent(Component component, int row, int col, GridBagConstraints gbc)
{
        gbc.gridx = col;
        gbc.gridy = row;
        add(component, gbc);
    }
   public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            new StudentForm().setVisible(true);
        });
    }
}
```

OUTPUT:



CONCLUSION

Thus, the program successfully demonstrated the creation of a structured data entry form using various Java Swing components.

LAB 7

OBJECTIVE: To write a java code to connect database and perform insert, select, delete and update operations (CRUD) in database table.

SOURCE CODE

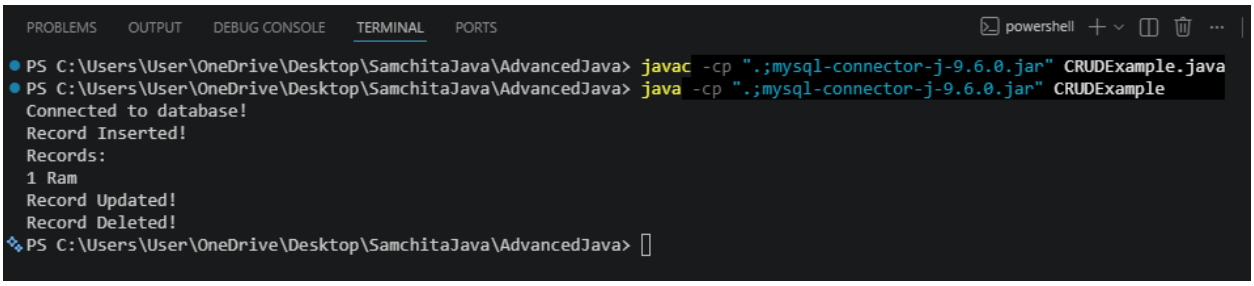
```
import java.sql.*;
public class CRUDExample {
   public static void main(String[] args) {
      try {
         // 1. Load Driver
         Class.forName("com.mysql.cj.jdbc.Driver");

         // 2. Establish Connection
         Connection con = DriverManager.getConnection(
          "jdbc:mysql://localhost:3306/college", "root", "MERO@ms321");

         System.out.println("Connected to database!");

          // 3. INSERT
         String insertQuery = "INSERT INTO student VALUES (1, 'Ram')";
         Statement stmt = con.createStatement();
         stmt.executeUpdate(insertQuery);
         System.out.println("Record Inserted!");

         // 4. SELECT
         String selectQuery = "SELECT * FROM student";
         ResultSet rs = stmt.executeQuery(selectQuery);

         System.out.println("Records:");
         while(rs.next()) {
            System.out.println(rs.getInt("id") + " " + rs.getString("name"));
         }

         // 5. UPDATE
         String updateQuery = "UPDATE student SET name='Shyam' WHERE id=1";
         stmt.executeUpdate(updateQuery);
         System.out.println("Record Updated!");

         // 6. DELETE
         String deleteQuery = "DELETE FROM student WHERE id=1";
         stmt.executeUpdate(deleteQuery);
         System.out.println("Record Deleted!");
```

```
            // Close connection
            con.close();

        } catch(Exception e) {
            System.out.println(e);
        }
    }
}
```

SQL QUERY

```
CREATE DATABASE college;
USE college;

CREATE TABLE student(
  id INT,
  name VARCHAR(50)
);
SHOW DATABASES;
SHOW TABLES;
```

OUTPUT



CONCLUSION

Thus, the program successfully demonstrated database connectivity and CRUD operations using JDBC in Java.

LAB 8

OBJECTIVE: To write a program using Connection oriented Socket Programming that can read write on both side.

THEORY

Socket programming enables communication between two systems over a network. TCP (Transmission Control Protocol) is connection-oriented, meaning a connection is established before data transfer.

SOURCE CODE:

i) ChatServer.java

```
package SocketChatApp;
import java.io.*;
import java.net.*;

public class ChatServer {
   public static void main(String[] args) {
      ServerSocket serverSocket = null;
      Socket socket = null;

      try {
         serverSocket = new ServerSocket(5000);
         System.out.println("Server started. Waiting for client...");

         socket = serverSocket.accept();
         System.out.println("Client connected!");

         BufferedReader input = new BufferedReader(|
               new InputStreamReader(socket.getInputStream()));

         PrintWriter output = new PrintWriter(socket.getOutputStream(), true);
         BufferedReader keyboard = new BufferedReader(
               new InputStreamReader(System.in));

         String clientMsg, serverMsg;
         while (true) {
            clientMsg = input.readLine();

           if (clientMsg == null || clientMsg.equalsIgnoreCase("bye")) {
               System.out.println("Connection closed.");
               break;
            }
```

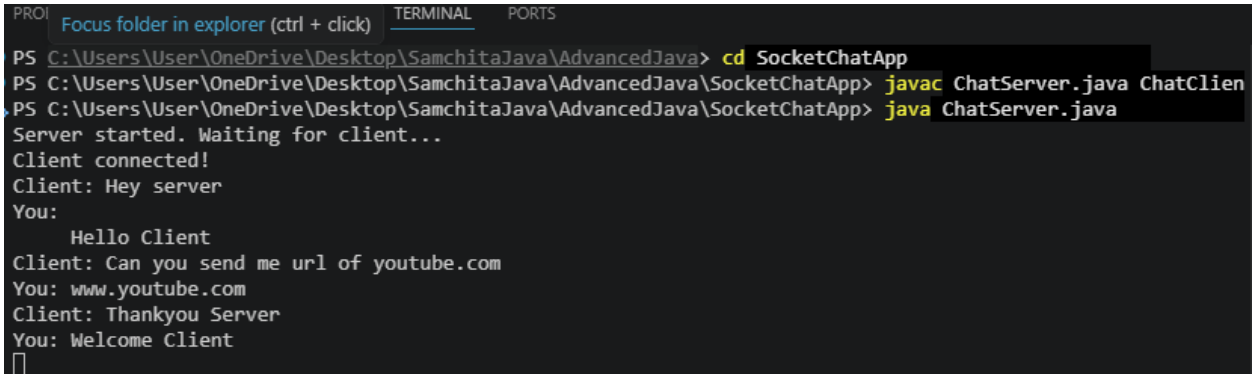
```
            System.out.println("Client: " + clientMsg);
            System.out.print("You: ");
            serverMsg = keyboard.readLine();
            output.println(serverMsg);
            if (serverMsg.equalsIgnoreCase("bye")) {
                System.out.println("Connection closed.");
                break;
            }
        }
    } catch (Exception e) {
        System.out.println(e);
    } finally {
        try {
            if (socket != null) socket.close();
            if (serverSocket != null) serverSocket.close();
        } catch (IOException e) {
            System.out.println(e);
        }
    }
  }
}
```

ii) ChatClient.java

```
package SocketChatApp;
import java.io.*;
import java.net.*;

public class ChatClient {|
  public static void main(String[] args) {
    Socket socket = null;
    BufferedReader input = null;
    PrintWriter output = null;
    BufferedReader keyboard = null;

    try {
        socket = new Socket("localhost", 5000);
        input = new BufferedReader(
                new InputStreamReader(socket.getInputStream()));

        output = new PrintWriter(socket.getOutputStream(), true);
       keyboard = new BufferedReader(
                new InputStreamReader(System.in));
```

```
            String serverMsg, clientMsg;
            System.out.println("Connected to server!");
            while (true) {
                System.out.print("You: ");
                clientMsg = keyboard.readLine();
                output.println(clientMsg);
                if (clientMsg.equalsIgnoreCase("bye")) {
                    System.out.println("Connection closed.");
                    break;
                }
                serverMsg = input.readLine();
                if (serverMsg == null || serverMsg.equalsIgnoreCase("bye")) {
                    System.out.println("Server closed connection.");
                    break;
                }
                System.out.println("Server: " + serverMsg);
            }
        } catch (Exception e) {
            System.out.println(e);
        } finally {
            try {
                if (keyboard != null) keyboard.close();
                if (input != null) input.close();
                if (output != null) output.close();
                if (socket != null) socket.close();
            } catch (IOException e) {
                System.out.println(e);
            }
        }
    }
}
```

OUTPUT



```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\User\OneDrive\Desktop\SamchitaJava\AdvancedJava> cd SocketChatApp
PS C:\Users\User\OneDrive\Desktop\SamchitaJava\AdvancedJava\SocketChatApp> javac ChatServer.java ChatClien
t.java
PS C:\Users\User\OneDrive\Desktop\SamchitaJava\AdvancedJava\SocketChatApp> java ChatClient.java
Connected to server!
You: Hey server
Server: Hello Client
You: Can you send me url of youtube.com
Server: www.youtube.com
You: Thankyou Server
Server: Welcome Client
You:

Ln 62, Col 2 (1864 selected)   Spaces: 4   UTF-8   CRLF   { } Java
```

CONCLUSION

Thus, the program successfully demonstrated two-way communication using connection-oriented socket programming in Java.

LAB 9

OBJECTIVE: To write client and server programs in which a server program accepts a radius of a circle from the client program that computes area, sends the computed area to the client program, and displays it by client program.

THEORY

This program demonstrates client-server communication using socket programming. The client sends the radius of a circle to the server. The server calculates the area using the formula:

$$A = \pi r^2$$

SOURCE CODE

i) AreaServer.java

```
package CircleArea;
import java.io.*;
import java.net.*;
public class AreaServer {|
   public static void main(String[] args) {
      try {
         ServerSocket serverSocket = new ServerSocket(5000);
         System.out.println("Server started. Waiting for client...");
         Socket socket = serverSocket.accept();
         System.out.println("Client connected!");
         BufferedReader input = new BufferedReader(
               new InputStreamReader(socket.getInputStream()));
         PrintWriter output = new PrintWriter(socket.getOutputStream(), true);

         String radiusStr = input.readLine();
         double radius = Double.parseDouble(radiusStr);

         double area = 3.1416 * radius * radius;

         output.println(area);
         System.out.println("Area calculated and sent to client: " + area);

         socket.close();
         serverSocket.close();
      } catch (Exception e) {
         System.out.println(e);
      }
   }
}
```

ii) AreaClient.java

```
package CircleArea;
import java.io.*;
import java.net.*;

public class AreaClient {
   public static void main(String[] args) {

      try {
         Socket socket = new Socket("localhost", 5000);
         BufferedReader input = new BufferedReader(
               new InputStreamReader(socket.getInputStream()));

         PrintWriter output = new PrintWriter(socket.getOutputStream(), true);
         BufferedReader keyboard = new BufferedReader(
               new InputStreamReader(System.in));

         // Step 1: Take radius from user
         System.out.print("Enter radius: ");
         String radius = keyboard.readLine();
        // Step 2: Send radius to server
         output.println(radius);
         // Step 3: Receive area from server
         String area = input.readLine();
         System.out.println("Area of circle is: " + area);
         // Close connections
         socket.close();
      } catch (Exception e) {
         System.out.println(e);
      }
   }
}
```

OUTPUT

AreaServer.java

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\User\OneDrive\Desktop\SamchitaJava\AdvancedJava> cd CircleArea
PS C:\Users\User\OneDrive\Desktop\SamchitaJava\AdvancedJava\CircleArea> javac AreaServer.java AreaClient.j
PS C:\Users\User\OneDrive\Desktop\SamchitaJava\AdvancedJava\CircleArea> java AreaServer.java
Server started. Waiting for client...
Client connected!
Area calculated and sent to client: 153.9384
PS C:\Users\User\OneDrive\Desktop\SamchitaJava\AdvancedJava\CircleArea>
```

AreaClient.java

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\User\OneDrive\Desktop\SamchitaJava\AdvancedJava> cd CircleArea
PS C:\Users\User\OneDrive\Desktop\SamchitaJava\AdvancedJava\CircleArea> javac AreaServer.java AreaClient.
ava
PS C:\Users\User\OneDrive\Desktop\SamchitaJava\AdvancedJava\CircleArea> java  AreaClient.java
Enter radius: 7
Area of circle is: 153.9384
PS C:\Users\User\OneDrive\Desktop\SamchitaJava\AdvancedJava\CircleArea>
```

CONCLUSION

Thus, the program successfully demonstrated client-server communication and remote computation using socket programming.

LAB 10

OBJECTIVE: To write program for client server application using TCP and UDP.

THEORY

This lab demonstrates communication using both TCP and UDP protocols. TCP is connection-oriented protocol that provides reliable communication and uses Socket and ServerSocket whereas UDP is a connectionless protocol that is faster but less reliable and uses DatagramSocket and DatagramPacket. In both programs, messages are exchanged between client and server systems.

SOURCE CODE:

a) **TCPServer.java**

```
import java.io.*;
import java.net.*;

public class TCPServer {
   public static void main(String[] args) {
      try {
         ServerSocket serverSocket = new ServerSocket(5000);
         System.out.println("TCP Server started... Waiting for client");

         Socket socket = serverSocket.accept();
         System.out.println("Client connected!");

         BufferedReader input = new BufferedReader(
               new InputStreamReader(socket.getInputStream()));

         PrintWriter output = new PrintWriter(socket.getOutputStream(), true);
         String message = input.readLine();
         System.out.println("Client: " + message);
         output.println("Message received by server");

         socket.close();
         serverSocket.close();
      } catch (Exception e) {
         System.out.println(e);
      }
   }
}
```

b) TCPClient.java

```
import java.io.*;
import java.net.*;


public class TCPClient {
   public static void main(String[] args) {
      try {
         Socket socket = new Socket("localhost", 5000);
         BufferedReader input = new BufferedReader(
               new InputStreamReader(socket.getInputStream()));


         PrintWriter output = new PrintWriter(socket.getOutputStream(), true);
        BufferedReader keyboard = new BufferedReader(
               new InputStreamReader(System.in));

         System.out.print("Enter message: ");
         String msg = keyboard.readLine();

         output.println(msg);
         String response = input.readLine();
         System.out.println("Server: " + response);

        socket.close();

      } catch (Exception e) {
         System.out.println(e);
      }
   }
}
```

c) UDP Server

```
import java.net.*;
public class UDPServer {
   public static void main(String[] args) {
      try {
         DatagramSocket socket = new DatagramSocket(5000);

         byte[] receiveData = new byte[1024];

        DatagramPacket receivePacket =
               new DatagramPacket(receiveData, receiveData.length);
         socket.receive(receivePacket);
```

```
            String message = new String(receivePacket.getData(), 0, receivePacket.getLength());
            System.out.println("Client: " + message);

            String response = "Message received (UDP Server)";
            byte[] sendData = response.getBytes();

            InetAddress ip = receivePacket.getAddress();|
            int port = receivePacket.getPort();
            DatagramPacket sendPacket =
                    new DatagramPacket(sendData, sendData.length, ip, port);


            socket.send(sendPacket);
            socket.close();
        } catch (Exception e) {
           System.out.println(e);
        }
    }
}
```

d) UDP Client

```
import java.net.*;
import java.util.Scanner;

public class UDPClient {
    public static void main(String[] args) {


        try {
            DatagramSocket socket = new DatagramSocket();

            Scanner sc = new Scanner(System.in);
            System.out.print("Enter message: ");
            String msg = sc.nextLine();

            byte[] sendData = msg.getBytes();
            InetAddress ip = InetAddress.getByName("localhost");

            DatagramPacket sendPacket =
                    new DatagramPacket(sendData, sendData.length, ip, 5000);
            socket.send(sendPacket);

            byte[] receiveData = new byte[1024];
```

```
            DatagramPacket receivePacket =
                    new DatagramPacket(receiveData, receiveData.length);
            socket.receive(receivePacket);
            String response = new String(receivePacket.getData(), 0, receivePacket.getLength());
            System.out.println("Server: " + response);
            socket.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

OUTPUT

TCPServer

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\User\OneDrive\Desktop\SamchitaJava\AdvancedJava> cd TCP-UDP-SERVER
PS C:\Users\User\OneDrive\Desktop\SamchitaJava\AdvancedJava\TCP-UDP-SERVER> javac TCPClient.java TCPServe
PS C:\Users\User\OneDrive\Desktop\SamchitaJava\AdvancedJava\TCP-UDP-SERVER> java TCPClient.java
java.net.ConnectException: Connection refused: connect
PS C:\Users\User\OneDrive\Desktop\SamchitaJava\AdvancedJava\TCP-UDP-SERVER> java TCPClient.java
Enter message: Hey Server
Server: Message received by server
PS C:\Users\User\OneDrive\Desktop\SamchitaJava\AdvancedJava\TCP-UDP-SERVER>
```

TCPClient

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\User\OneDrive\Desktop\SamchitaJava\AdvancedJava\TCP-UDP-SERVER> javac TCPClient.java TCPServe
r.java
PS C:\Users\User\OneDrive\Desktop\SamchitaJava\AdvancedJava\TCP-UDP-SERVER> java TCPServer.java
TCP Server started... Waiting for client
Client connected!
Client: Hey Server
PS C:\Users\User\OneDrive\Desktop\SamchitaJava\AdvancedJava\TCP-UDP-SERVER>
```

UDPServer

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\User\OneDrive\Desktop\SamchitaJava\AdvancedJava> cd TCP-UDP-SERVER
PS C:\Users\User\OneDrive\Desktop\SamchitaJava\AdvancedJava\TCP-UDP-SERVER> javac UDPServer.java UDPClien
PS C:\Users\User\OneDrive\Desktop\SamchitaJava\AdvancedJava\TCP-UDP-SERVER> java UDPServer.java
Client: Hey there!!!
PS C:\Users\User\OneDrive\Desktop\SamchitaJava\AdvancedJava\TCP-UDP-SERVER>
```

UDPClient

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\User\OneDrive\Desktop\SamchitaJava\AdvancedJava> cd TCP-UDP-SERVER
PS C:\Users\User\OneDrive\Desktop\SamchitaJava\AdvancedJava\TCP-UDP-SERVER> javac UDPServer.java UDPClien
t.java
PS C:\Users\User\OneDrive\Desktop\SamchitaJava\AdvancedJava\TCP-UDP-SERVER> java UDPClient.java
Enter message: Hey there!!!
Server: Message received (UDP Server)
PS C:\Users\User\OneDrive\Desktop\SamchitaJava\AdvancedJava\TCP-UDP-SERVER>
```

CONCLUSION

Thus, the program successfully illustrated communication using both TCP and UDP networking protocols in Java.

LAB 11

OBJECTIVE: To write a program that illustrate Handling HTTP Request and Response using servlets.

THEORY

A Servlet is a Java program that runs on a web server and handles client requests.
In this program:

- HttpServletRequest is used to receive data from the client.
- HttpServletResponse is used to send responses back to the browser.
- doGet() method handles HTTP GET requests.

The servlet reads a parameter from the URL and dynamically generates an HTML response.

SOURCE CODE

1) Servlet.java

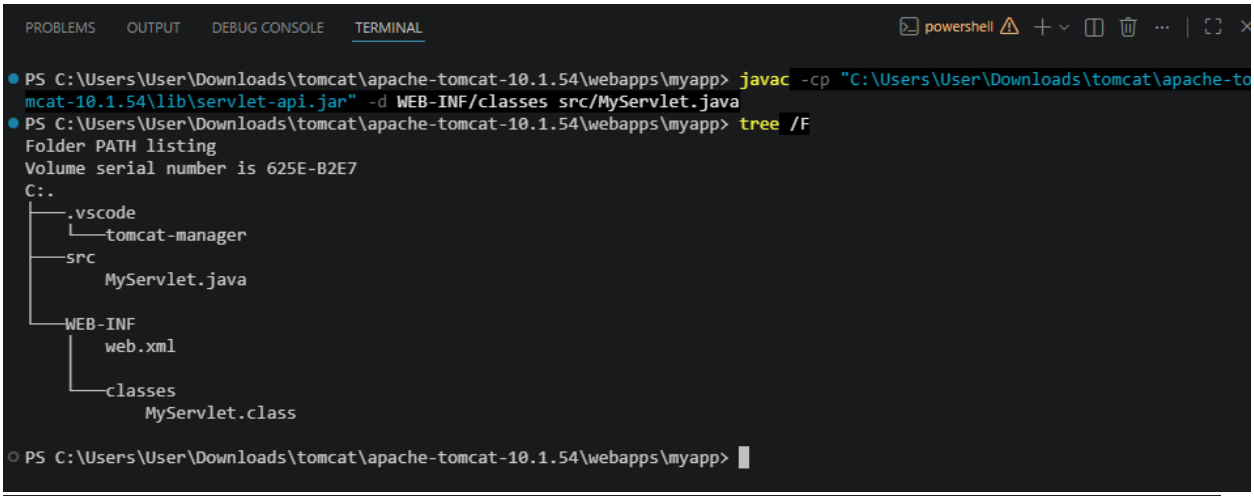
```
import java.io.IOException;
import java.io.PrintWriter;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

public class MyServlet extends HttpServlet {
   protected void doGet(HttpServletRequest request, HttpServletResponse response)
         throws ServletException, IOException {


            response.setContentType("text/html");

            PrintWriter out = response.getWriter();

            String name = request.getParameter("name");

            if (name == null || name.equals("")) {
         name = "Guest";
      }

      // Write response to browser
      out.println("<html><body>");
      out.println("<h2>HTTP Request and Response Example</h2>");
      out.println("<p>Hello, " + name + "</p>");
      out.println("</body></html>");
```

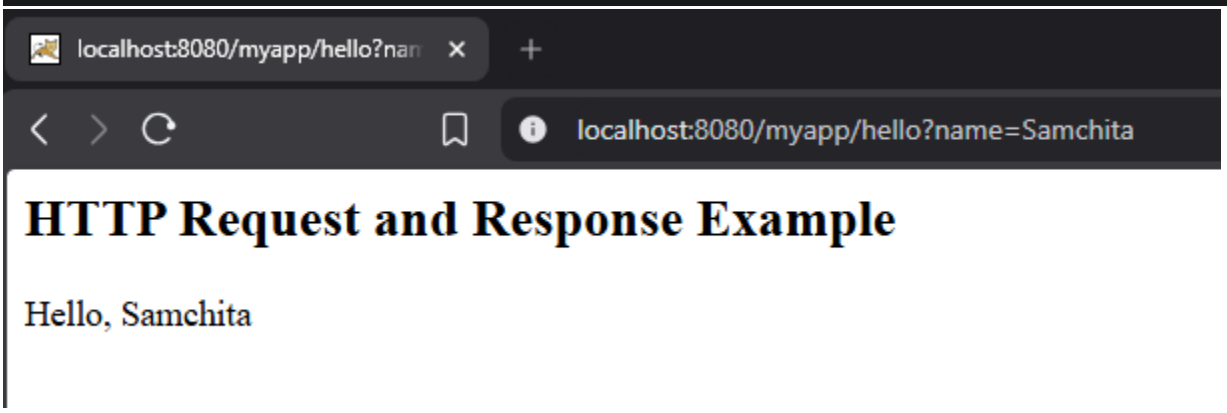
```
    }
}
```

2) web.xml

```
<web-app>
    <servlet>
        <servlet-name>MyServlet</servlet-name>
        <servlet-class>MyServlet</servlet-class>
    </servlet>

    <servlet-mapping>
        <servlet-name>MyServlet</servlet-name>
        <url-pattern>/hello</url-pattern>
    </servlet-mapping>

</web-app>
```

OUTPUT





CONCLUSION

Thus, the program successfully demonstrated handling HTTP requests and responses using Java Servlets.

LAB 12

OBJECTIVE: To write a simple JSP program to display “Tribhuvan University” 10 times.

THEORY

JSP (Java Server Pages) allows embedding Java code inside HTML. This program uses:

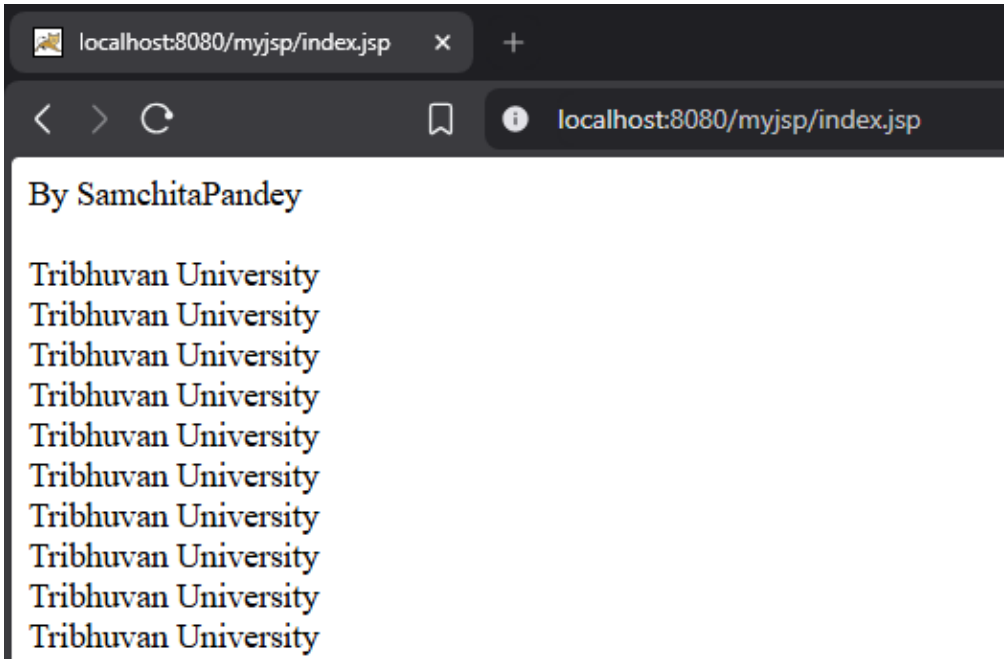
- Scriptlet tags <% %> for Java code execution.
- A loop to print “Tribhuvan University” ten times dynamically.

SOURCE CODE:

index.jsp

```
<%
out.println("By SamchitaPandey <br> <br>");
for(int i=1; i<=10; i++)
{
%>
Tribhuvan University <br>
<%
}

%>
```

OUTPUT



CONCLUSION

Thus, the program successfully demonstrated the use of JSP scriptlet tags and loops for dynamic content generation.

LAB 13

OBJECTIVE: To write a program to create a JSP web form to take input of a student and submit it to second JSP file which may simply print the values of form submission.

SOURCE CODE

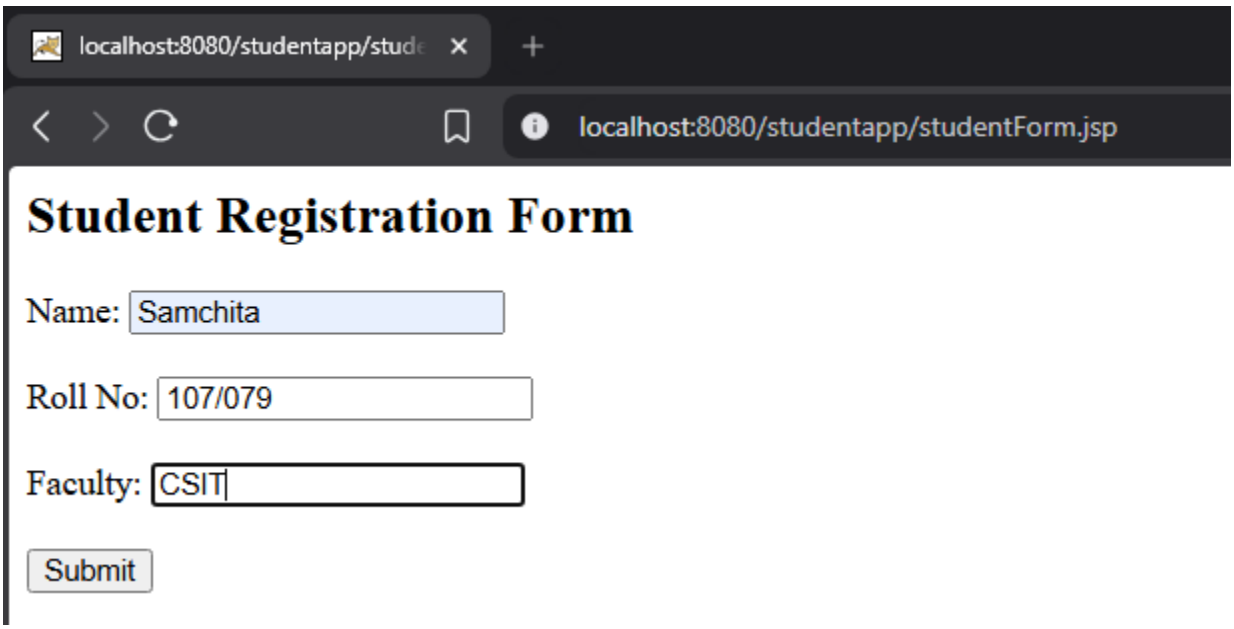
1) studentForm.jsp

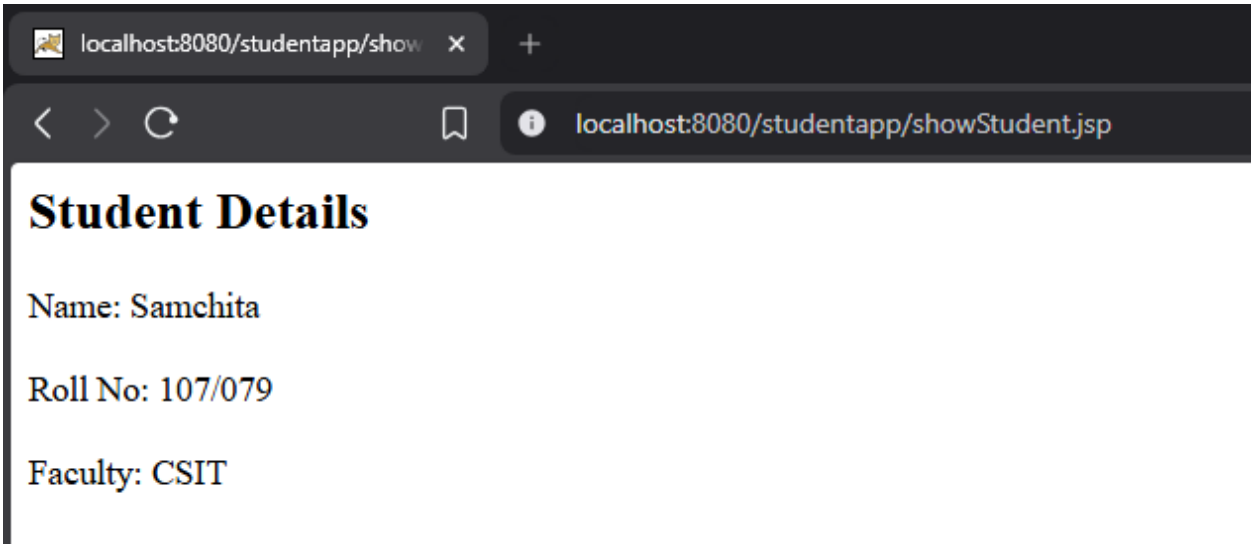
```
<html>
<body>

<h2>Student Registration Form</h2>
<form action="showStudent.jsp" method="post">
Name: <input type="text" name="name"><br><br>
Roll No: <input type="text" name="roll"><br><br>
Faculty: <input type="text" name="faculty"><br><br>

<input type="submit" value="Submit">
</form>

</body>
</html>
```

2) showStudent.jsp

```
<%
String name = request.getParameter("name");
String roll = request.getParameter("roll");
String faculty = request.getParameter("faculty");
%>

<html>
<body>

<h2>Student Details</h2>

Name: <%= name %> <br><br>
Roll No: <%= roll %> <br><br>
Faculty: <%= faculty %> <br><br>

</body>
</html>
```

OUTPUT





CONCLUSION

Thus, the program successfully demonstrated form submission and data retrieval between JSP pages.

LAB 14

OBJECTIVE: To write a program using RMI to find sum and difference of two numbers. Methods sum and difference should be invoked from some remote machine.

THEORY:
RMI (Remote Method Invocation) allows methods of remote objects to be invoked from another machine. In this program:

- The interface defines remote methods.
- The server registers the remote object.
- The client accesses the remote methods using Naming.lookup().

SOURCE CODE:

1) Calculator.java
```
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface Calculator extends Remote {
   public int sum(int a, int b) throws RemoteException;
   public int difference(int a, int b) throws RemoteException;
}
```

2) CalculatorImp1.java
```
import java.rmi.Remote;
import java.rmi.server.UnicastRemoteObject;
import java.rmi.RemoteException;

public class CalculatorImpl extends UnicastRemoteObject implements Calculator {
   public CalculatorImpl() throws RemoteException {
      super();
   }

   public int sum(int a, int b) throws RemoteException {
      return a + b;
   }

   public int difference(int a, int b) throws RemoteException {
     return a - b;
   }
}
```

3) Client.java
```
import java.rmi.Remote;
```

```
import java.rmi.RemoteException;
import java.rmi.Naming;

public class Client {
    public static void main(String[] args) {
        try {
            Calculator obj = (Calculator) Naming.lookup("rmi://localhost/CalculatorService");

            int a = 10;
            int b = 5;

            System.out.println("Sum: " + obj.sum(a, b));
            System.out.println("Difference: " + obj.difference(a, b));

        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

4) Server.java

```
public class Server {
    public static void main(String[] args) {
        try {
            Naming.rebind("rmi://localhost/CalculatorService", new CalculatorImpl());
            System.out.println("Server is ready...");

            CalculatorImpl obj = new CalculatorImpl();
            Naming.rebind("rmi://localhost/CalculatorService", obj);

            System.out.println("Server is ready...");

        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

OUTPUT

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL

PS C:\Users\User\OneDrive\Desktop\SamchitaJava> cd C:\Users\User\OneDrive\Desktop\SamchitaJava\AdvancedJav
a\lab14RMI
PS C:\Users\User\OneDrive\Desktop\SamchitaJava\AdvancedJava\lab14RMI> java Client
Sum: 15
Difference: 5
PS C:\Users\User\OneDrive\Desktop\SamchitaJava\AdvancedJava\lab14RMI> tree /F
Folder PATH listing
Volume serial number is 625E-B2E7
C:.
    Calculator.class
    Calculator.java
    CalculatorImpl.class
    CalculatorImpl.java
    Client.class
    Client.java
    Server.class
    Server.java
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL

PS C:\Users\User\OneDrive\Desktop\SamchitaJava> cd C:\Users\User\OneDrive\Desktop\SamchitaJava\AdvancedJav
a\lab14RMI
PS C:\Users\User\OneDrive\Desktop\SamchitaJava\AdvancedJava\lab14RMI> java Client
Sum: 15
Difference: 5
PS C:\Users\User\OneDrive\Desktop\SamchitaJava\AdvancedJava\lab14RMI>
```

CONCLUSION

Thus, the program successfully demonstrated remote method invocation using Java RMI architecture.

LAB 15

OBJECTIVE: To implement program that illustrate the example of servlet and database.

THEORY

This program demonstrates integration of Servlets with a MySQL database using JDBC. The servlet:

1. Receives form data from the user
2. Connects to the database
3. Inserts records into the database table using PreparedStatement

PreparedStatement is used because it improves security and efficiency.

SOURCE CODE

1) **UserServlet.java**

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;
import java.sql.*;

public class UserServlet extends HttpServlet {
    public void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        // Get form data
        String name = request.getParameter("name");
        String email = request.getParameter("email");

        try {
            // Load MySQL driver
            Class.forName("com.mysql.cj.jdbc.Driver");

            // Database connection
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/testdb",
                "root",
                "MERO@ms321"
            );

            // Insert query
```

```
            PreparedStatement ps = con.prepareStatement(
                "INSERT INTO users(name, email) VALUES(?, ?)"
            );

            ps.setString(1, name);
            ps.setString(2, email);

            int i = ps.executeUpdate();
            if(i > 0) {
                out.println("Data inserted successfully!");
            }
            con.close();
        } catch(Exception e) {
            out.println(e);
        }
    }
}
```

2) Web.xml

```
<web-app>

    <servlet>
        <servlet-name>UserServlet</servlet-name>
        <servlet-class>UserServlet</servlet-class>
    </servlet>

    <servlet-mapping>
        <servlet-name>UserServlet</servlet-name>
        <url-pattern>/UserServlet</url-pattern>
    </servlet-mapping>

</web-app>
```

3) index.html

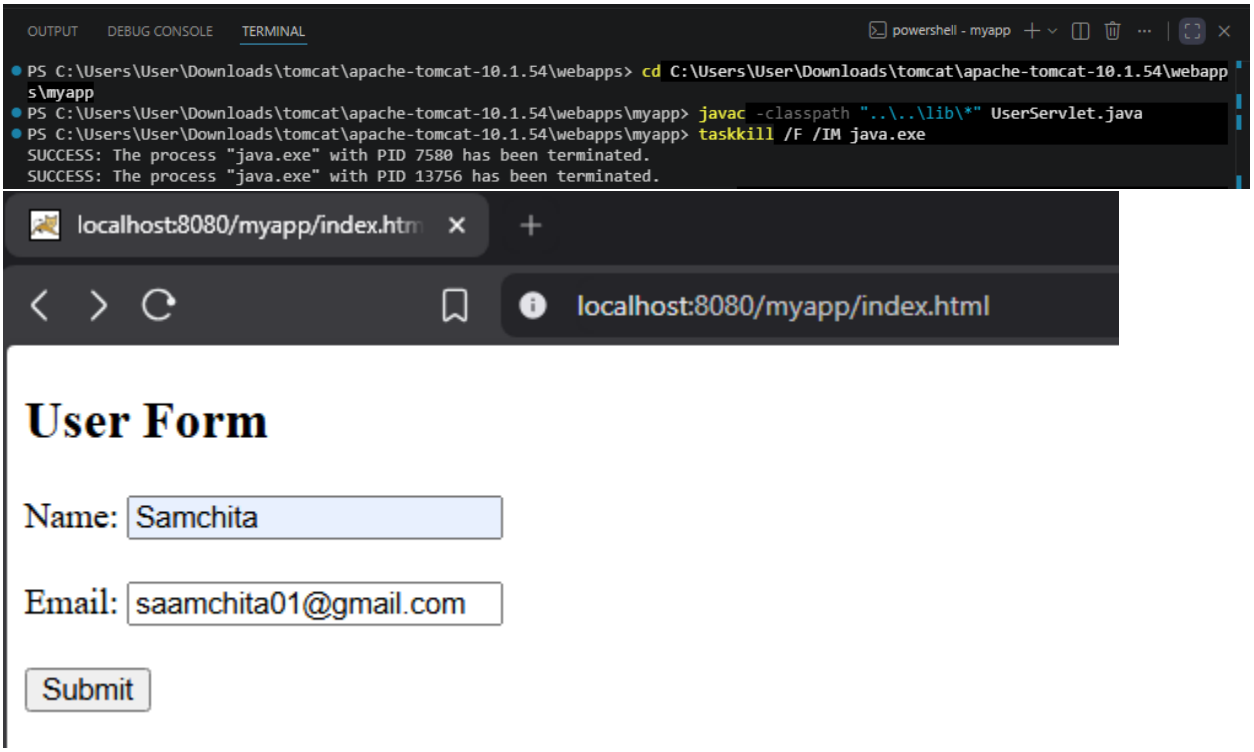
```
<!DOCTYPE html>
<html>
<body>

<h2>User Form</h2>

<form action="UserServlet" method="post">
```

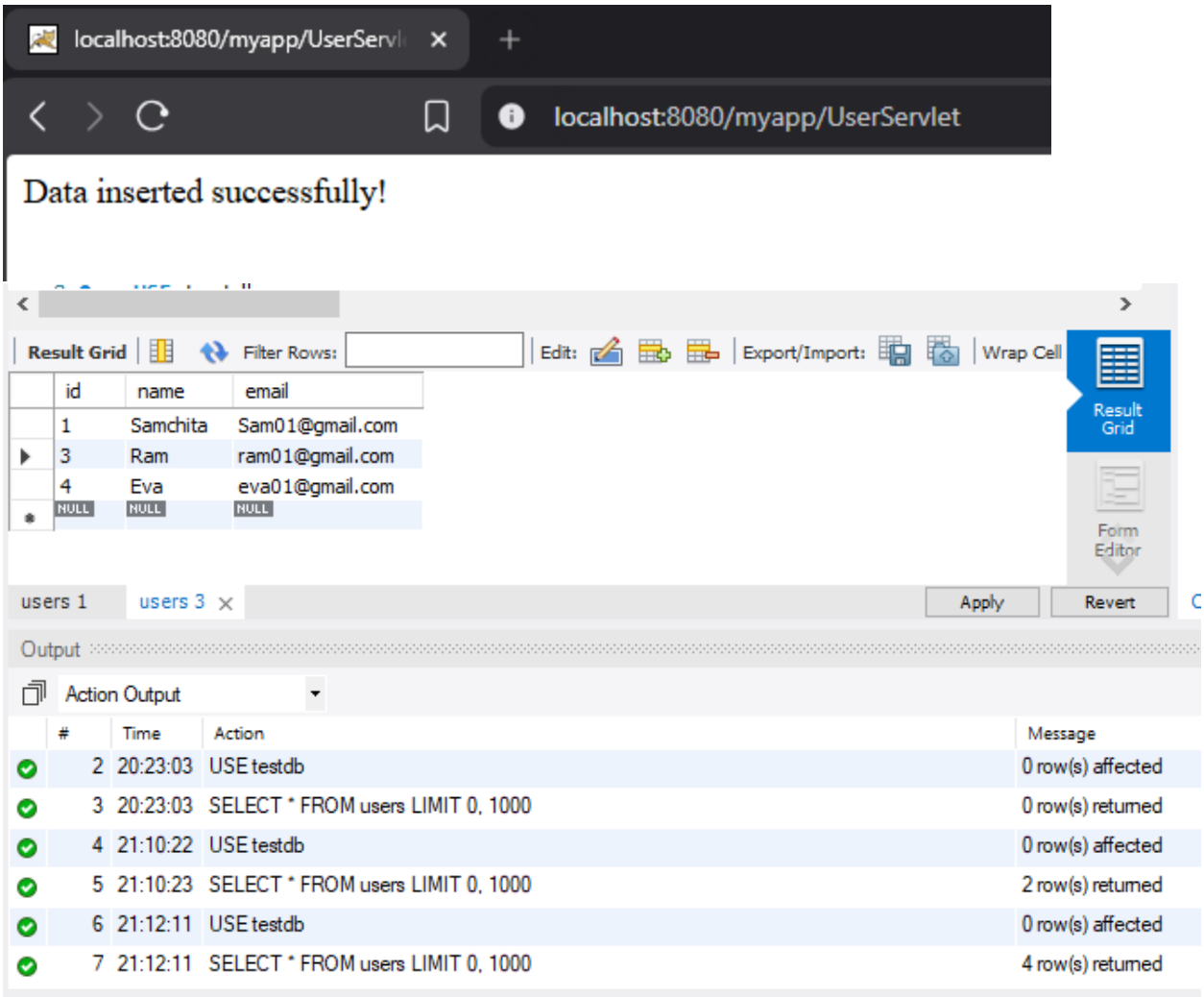
```
    Name: <input type="text" name="name"><br><br>
    Email: <input type="text" name="email"><br><br>
    <input type="submit" value="Submit">
</form>

</body>
</html>
```

4) SQL QUERY

```
CREATE DATABASE testdb;
USE testdb;
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(100)
);

USE testdb;
SELECT * FROM users;
```

OUTPUT





CONCLUSION

Thus, the program successfully demonstrated database connectivity and data insertion using Servlets and JDBC.

LAB 16

OBJECTIVE: To write a program to illustrate example of criplet tag, declaration tag, and expression tag.

THEORY

JSP provides different types of tags for embedding Java code into web pages.

1. Declaration Tag
It is used to declare variables and methods.
<%! %>

2. Scriptlet Tag
It is used to write Java statements**.**
<% %>

3. Expression Tag
It is used to display values directly in the browser.
<%= %>

This program demonstrates the use of all three JSP tags.

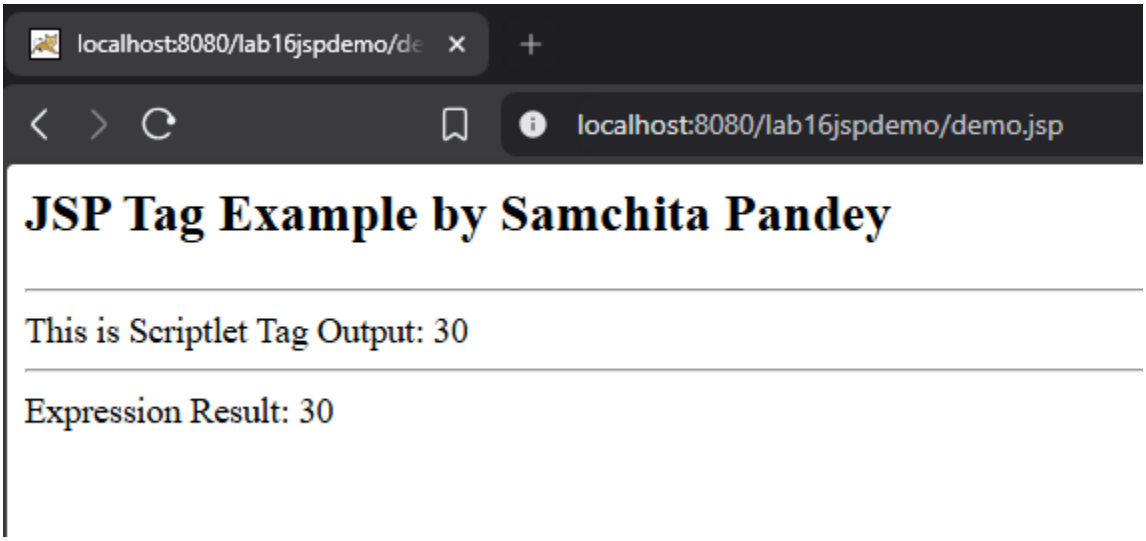
SOURCE CODE

1)Index.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8" %>
<html>
<head>
   <title>JSP Demo Home</title>
</head>
<body>

<h2>Welcome to JSP Tag Demonstration</h2>

<p>This project demonstrates:</p>

<ul>
   <li>Declaration Tag (&lt;%! %&gt;)</li>
   <li>Scriptlet Tag (&lt;% %&gt;)</li>
   <li>Expression Tag (&lt;%= %&gt;)</li>
</ul>

<!-- Link to demo.jsp -->
<a href="demo.jsp">Click here to see JSP Tags Example</a>
```

```
</body>
</html>
```

OUTPUT



CONCLUSION

Thus, the program successfully demonstrated declaration, scriptlet, and expression tags in JSP.